

SEGFAULT 2026

PathWitness

Remembering what the program already told LLVM

Team: DietCode

The idea

Compilers often know more about a program than they manage to use. A condition on one branch may establish a fact that makes a later condition unnecessary, but that fact can be lost or become awkward to represent as the program moves through LLVM's optimization pipeline.

We want to explore one small but concrete version of this problem: remembering when two values are known to be the same or known to be different along a path, and using that information to remove code which cannot execute.

Consider this function:

```
extern void side_effect(void);

void f(long a, long b, long c) {
    if (a != b)
        if (b == c)
            if (a == c)
                side_effect();
}
```

The call is impossible. If $b == c$ and $a == c$, then $a == b$; that contradicts the first condition, $a != b$. A person can see the contradiction immediately. In our experiment with LLVM 22.1.8, the call is still present after `-O3`.

This is the starting point for PathWitness.

Where the gap appears

LLVM already has several forms of path-sensitive reasoning. In particular, its `ConstraintElimination` pass collects comparisons from dominating branches and uses them to simplify later comparisons. It is good at facts that fit naturally into a system of linear inequalities. For example, from $a > b$ and $b > c$, it can reason about $a > c$.

A general $a != b$ fact is different. It means that either $a < b$ or $a > b$; it is not one continuous linear region. The current LLVM source still has an explicit note that non-equality facts are not supported as stored facts in this pass, and such a fact is skipped. LLVM can answer some equality and non-equality questions, but it cannot keep the general fact in the form needed by the example above.

There is also a practical pipeline issue. Nested source-level `if` statements do not necessarily remain nested branches. Earlier optimizations may combine them into a Boolean expression made of `and` and `or`. The information has not disappeared logically, but its shape has changed. We are interested in the boundary between those representations: what was known on the original path, what remains visible after simplification, and how a later optimization can use it cheaply.

This places the project in the LLVM middle end, between control-flow analysis, value reasoning, and the design of optimization pipelines. It is not a proposal for a new language or backend. It is an attempt to make an existing optimizer carry one useful kind of knowledge a little further.

What we plan to try

Our initial approach is to place a small relational layer beside LLVM's existing linear constraint reasoning. It would remember groups of values known to be equal and a bounded set of “different from” relationships between those groups. If two groups later merge because of an equality, their relationships move with them. This is enough to follow chains such as:

$$a \neq b \quad + \quad b == c \quad \implies \quad a \neq c$$

We do not intend to put a general theorem prover or SMT solver inside the normal compiler pipeline. The useful question is whether a deliberately limited structure can catch real cases while remaining predictable in compile time and conservative whenever it does not understand something.

We would also like the pass to say why it changed the program. For the example above, an optimization remark could show the two earlier facts and the contradiction they establish. This would make the optimization easier to debug and give us a direct way to inspect whether our reasoning is doing what we intended.

Goal

The strongest result would be a small LLVM change that removes a measured class of redundant checks with negligible compile-time cost, explains each proof, and is clear enough to send upstream. A useful conservative result would still be a well-tested plugin and a concrete study of when disequality tracking helps and when it does not.

Either way, the project has a simple question at its centre:

When the program has already proved that a path is impossible, can LLVM remember enough to agree?